



UNIVERSIDAD SANTA MARIA
FACULTAD INGENIERIA Y ARQUITECTURA - SISTEMAS

Proyecto de Ingeniería de Software II — Ciberseguridad

Plataforma CTF con Contenedores Dinámicos:
Diseño, Implementación y Validación de un Orquestador de
Docker con Aislamiento por Usuario y Retos OWASP Top 10

Integrantes:

Agostini, Pedro
Castellano, Daniel
Guillén, Arantza
Pérez, Samuel
Peña, José

Profesor: Alexander Moya

Caracas, Venezuela, septiembre de 2026

Índice

Introducción	5
Capítulo 1. Definición del Proyecto	7
1.1 Descripción general	7
1.2 Problemas a resolver	7
1.3 Objetivos	8
1.3.1 Objetivo general	8
1.3.2 Objetivos específicos	8
1.4 Alcance	8
Capítulo 2. Marco Teórico	10
2.1 Competencias Capture The Flag como herramienta pedagógica	10
2.2 Contenedores Linux: namespaces y cgroups	10
2.3 Docker Engine API y socket Unix	10
2.4 OWASP Top 10 (2021)	11
2.5 Arquitectura ASGI, WebSockets y emulación de terminal	11
2.6 Principios de diseño seguro	11
2.7 Metodologías iterativas e incrementales	12
Capítulo 3. Análisis de Vulnerabilidad	13
3.1 Modelo de amenazas de la plataforma	13
3.2 Vulnerabilidades pedagógicas: los diez retos OWASP	14
Capítulo 4. Diseño de la Solución	16
4.1 Arquitectura general	16
4.2 Componentes	17
4.2.1 Driver de Docker (docker_client.py)	17
4.2.2 Vistas HTTP (views.py)	17
4.2.3 Consumidor WebSocket (consumers.py)	17
4.2.4 Catálogo de retos (challenges.py)	17
4.2.5 Watchdog (watchdog.py, apps.py)	17
4.3 Flujos principales	18

4.4 Mecanismos de aislamiento y endurecimiento	18
4.5 Decisiones de diseño y alternativas descartadas	19
Capítulo 5. Metodología de Desarrollo	21
5.1 Enfoque adoptado	21
5.2 Herramientas y entorno	21
Capítulo 6. Implementación y Pruebas	22
6.1 Stack tecnológico	22
6.2 Implementación de los componentes clave	22
6.2.1 Hijacking del exec	22
6.2.2 Doble criterio de destrucción	23
6.2.3 Terminación por sesión	23
6.3 Estrategia y niveles de prueba	23
6.4 Pruebas unitarias automatizadas	24
6.5 Pruebas de aislamiento	25
6.6 Pruebas de estrés y de carga	26
Capítulo 7. Resultados	29
7.1 Resultados cuantitativos	29
7.2 Resultados cualitativos	30
7.3 Defectos detectados por la campaña de validación	30
Capítulo 8. Conclusiones y Mejoras Futuras	32
8.1 Conclusiones	32
8.2 Limitaciones	33
8.3 Mejoras futuras	33
Referencias	35
Anexos	37
Anexo A. Pruebas unitarias	37
Anexo B. Catálogo de retos y banderas dinámicas	38
Anexo C. Invariantes de la base de datos	39
Anexo D. API HTTP y ciclo de vida de la instancia	40

Anexo E. Límites enviados al kernel y panel de administración	41
Anexo F. Destrucción automática de instancias	42
Anexo G. Pruebas de humo	43
Anexo H. Pruebas de estrés: bomba de procesos	45
Anexo I. Un estudiante intenta alcanzar al otro	46
Anexo J. Perfil completo de la plataforma	47
Anexo K. Prueba de carga con veinte estudiantes	48

Introducción

La formación práctica en ciberseguridad exige entornos donde el estudiante pueda ejecutar ataques reales sobre software deliberadamente vulnerable sin poner en riesgo la infraestructura de la institución ni a los demás participantes. Las competencias de tipo Capture The Flag (CTF) son el formato más extendido para este propósito, porque convierten la explotación de una vulnerabilidad en un objetivo concreto y verificable: obtener una cadena secreta, la *flag*, que solo se alcanza al comprometer la falla (Chapman et al., 2014).

Sin embargo, montar un laboratorio de este tipo presenta un problema de ingeniería no trivial. Cada estudiante necesita un entorno propio, aislado de los demás, con límites de recursos que impidan que un error o un abuso (un bucle infinito, una *fork bomb*, etc.) degrade al servidor compartido, y con un ciclo de vida automático que libere recursos cuando el entorno deja de usarse. Los contenedores Linux, y en particular Docker, ofrecen la base técnica para resolverlo, pero su uso seguro en un escenario multiusuario requiere decisiones deliberadas de diseño.

Este trabajo documenta el diseño, la implementación y la validación de Orquestador-daemon, una plataforma CTF con contenedores dinámicos desarrollada como proyecto de la asignatura Ingeniería de Software II. La plataforma permite que cada usuario autenticado despliegue bajo demanda un contenedor aislado con uno de diez retos alineados al OWASP Top 10 (2021), interactúe con él mediante una consola en el navegador, superando el reto al ingresar la *flag* correcta. El sistema destruye la instancia de forma automática por inactividad o por vencimiento de su vida máxima.

El documento se estructura en ocho capítulos. El capítulo 1 define el proyecto, los problemas que resuelve, sus objetivos y su alcance. El capítulo 2 presenta el marco teórico. El capítulo 3 desarrolla el análisis de vulnerabilidad, tanto de las fallas pedagógicas que los retos exponen como de los vectores de ataque contra la propia plataforma. El capítulo 4 describe el diseño de la solución y la propuesta técnica adoptada. El capítulo 5 expone la metodología de desarrollo. El capítulo 6 detalla la implementación, las herramientas empleadas y la campaña de pruebas (unitarias, de humo, de estrés y de carga) realizada sobre el sistema. El capítulo 7

resume los resultados y los defectos detectados durante la validación, y el capítulo 8 presenta las conclusiones, las limitaciones del sistema y las mejoras futuras.

Capítulo 1. Definición del Proyecto

1.1 Descripción general

Orquestador-daemon es una aplicación web que actúa como orquestador de contenedores Docker para fines educativos. Su función central es recibir la solicitud de un estudiante autenticado, crear para él una red aislada y un contenedor con la imagen del reto elegido, exponer una consola interactiva sobre ese contenedor a través del navegador, validar la *flag* que el estudiante envía como también destruir el entorno cuando corresponde. El código fuente se encuentra en el repositorio público del proyecto (Castellanos et al., 2026).

El proyecto se planteó con tres retos técnicos obligatorios. El primero, de *backend*, exige que el orquestador se comunique directamente con el socket del daemon de Docker, sin usar el SDK oficial docker-py ni invocar el binario docker. El segundo, de *frontend*, exige integrar Xterm.js para ofrecer una terminal real en el navegador comunicada por WebSockets bidireccionales. El tercero, denominado *modo difícil*, exige aplicar límites estrictos de recursos mediante cgroups y namespaces, y destruir automáticamente las instancias por inactividad.

1.2 Problemas a resolver

El análisis del contexto identificó cinco problemas concretos que la plataforma debe resolver.

1. **Aislamiento entre estudiantes.** Un participante no debe poder alcanzar, observar ni interferir con el entorno de otro, ni salir a Internet desde el contenedor.
2. **Protección del servidor anfitrión.** Un proceso descontrolado dentro de un contenedor (agotamiento de memoria, saturación de CPU, *fork bomb*, etc.) no debe degradar al host ni a los demás contenedores.
3. **Ciclo de vida automático.** Los entornos abandonados deben destruirse sin intervención humana, incluso cuando un proceso en segundo plano simule actividad.
4. **Interacción en tiempo real.** La consola del navegador debe comportarse como una terminal real: soporte de secuencias ANSI, redimensionado, pegado seguro y baja latencia.

5. Integridad pedagógica. Cada reto debe resolverse únicamente explotando la vulnerabilidad prevista; no debe existir un atajo como leer la *flag* directamente del sistema de archivos.

1.3 Objetivos

1.3.1 Objetivo general

Diseñar, implementar y validar una plataforma CTF que despliegue contenedores Docker aislados bajo demanda para el usuario autenticado, con consola interactiva en el navegador, límites de recursos aplicados por el kernel y destrucción automática de instancias, cubriendo las diez categorías del OWASP Top 10 (2021).

1.3.2 Objetivos específicos

1. Implementar un cliente de la Docker Engine API sobre el socket Unix, incluido el *hijacking* de la conexión para el flujo de la terminal, sin bibliotecas de alto nivel.
2. Construir el orquestador con Django y Django Channels, con un modelo de datos que persista el estado de cada instancia.
3. Integrar Xterm.js con un consumidor WebSocket asíncrono que transporte bytes crudos entre el navegador y el proceso del contenedor.
4. Aplicar aislamiento de red por usuario y límites de memoria, CPU, procesos y descriptores mediante la configuración `HostConfig` del contenedor.
5. Implementar un *watchdog* que destruya instancias por inactividad y por vida máxima según la dificultad del reto.
6. Desarrollar diez retos con separación real de privilegios, uno por categoría del OWASP Top 10, con validación de *flags* en el servidor.
7. Verificar empíricamente el aislamiento, los límites y la robustez del sistema mediante una suite de pruebas automatizadas y una campaña de pruebas de humo, de estrés y de carga.

1.4 Alcance

El alcance del producto mínimo viable (MVP) incluye el orquestador completo, la consola interactiva, el aislamiento de red y recursos, el *watchdog* con doble criterio de destrucción, el catálogo de diez retos, la validación de *flags*, un panel de administración con

configuración editable en vivo y vista de instancias. La plataforma se ejecuta en Linux o en Windows con WSL2, porque depende del socket Unix de Docker.

Capítulo 2. Marco Teórico

2.1 Competencias Capture The Flag como herramienta pedagógica

Un CTF es un ejercicio en el que los participantes deben encontrar y explotar vulnerabilidades en sistemas preparados para ello, con el fin de obtener *flags* que demuestran el éxito del ataque. Vigna (2003) documentó tempranamente el valor de los ejercicios prácticos en entornos controlados para la enseñanza de seguridad de redes.

La condición esencial de un CTF educativo es que la *flag* solo sea alcanzable explotando la falla prevista. Si existe un atajo, el reto deja de medir la competencia que pretende enseñar. Este principio guió la decisión de separar privilegios dentro de cada contenedor, descrita en el capítulo 4.

2.2 Contenedores Linux: namespaces y cgroups

Un contenedor es un conjunto de procesos que el kernel de Linux aísla del resto del sistema mediante dos mecanismos complementarios. Los *namespaces* particionan los recursos visibles: un proceso dentro de un namespace de PID solo ve los procesos de ese namespace, uno dentro de un namespace de red solo ve sus propias interfaces y tablas de rutas, y así con los puntos de montaje, los identificadores de usuario y otros recursos (Kerrisk, 2024a). Los *control groups* (cgroups), por su parte, limitan y contabilizan el uso de recursos (memoria, tiempo de CPU, número de procesos) de un grupo de procesos (Kerrisk, 2024b). Docker combina ambos mecanismos con un sistema de archivos en capas para ofrecer contenedores ligeros y reproducibles (Merkel, 2014). La superficie de seguridad que resulta de ese modelo de aislamiento ha sido analizada de forma sistemática en la literatura (Combe et al., 2016; Sultan et al., 2019).

Las *capabilities* de Linux dividen los privilegios del superusuario en unidades independientes (Kerrisk, 2024c). Docker asigna por defecto un subconjunto reducido, pero la práctica recomendada en entornos multiusuario es retirarlas todas y devolver únicamente las estrictamente necesarias (Docker Inc., 2024b).

2.3 Docker Engine API y socket Unix

El daemon de Docker expone una API REST sobre el socket Unix. Toda operación del cliente oficial (crear redes, crear y arrancar contenedores, abrir procesos) corresponde a una

petición HTTP/1.1 sobre ese socket (Docker Inc., 2024a). Para las operaciones de flujo continuo, como adjuntar una terminal a un proceso, la API implementa un mecanismo de *hijacking* para ceder el control directo del socket a un flujo de datos bidireccional continuo.

2.4 OWASP Top 10 (2021)

El OWASP Top 10 es un documento de concienciación que enumera las diez categorías de riesgo más críticas en aplicaciones web, elaborado a partir de datos de más de 500 000 aplicaciones y de una encuesta a profesionales (OWASP Foundation, 2021). La edición 2021 incluye: A01 Pérdida de control de acceso, A02 Fallas criptográficas, A03 Inyección, A04 Diseño inseguro, A05 Configuración de seguridad incorrecta, A06 Componentes vulnerables y desactualizados, A07 Fallas de identificación y autenticación, A08 Fallas de integridad de software y datos, A09 Fallas de registro y monitoreo, A10 Falsificación de solicitudes del lado del servidor (SSRF). Cada categoría se asocia a debilidades del catálogo CWE de MITRE Corporation (2023), como CWE-89 para inyección SQL o CWE-502 para deserialización insegura. El catálogo de retos del proyecto cubre las diez categorías.

2.5 Arquitectura ASGI, WebSockets y emulación de terminal

El protocolo WebSocket, definido en la RFC 6455, establece un canal bidireccional persistente sobre una única conexión TCP, iniciado con un *handshake* HTTP (Fette y Melnikov, 2011). En el ecosistema Python, la especificación ASGI generaliza la interfaz WSGI para soportar protocolos de larga duración de forma asíncrona (Django Software Foundation, 2024b). Django Channels implementa ASGI para Django (Django Software Foundation, 2024c) e introduce los *consumers*, clases que manejan conexiones WebSocket; Daphne es el servidor ASGI de referencia (Django Software Foundation, 2024a).

Xterm.js es un emulador de terminal escrito en TypeScript que interpreta secuencias de escape ANSI en el navegador y expone eventos de entrada y redimensionado (Xterm.js Contributors, 2024).

2.6 Principios de diseño seguro

Saltzer y Schroeder (1975) formularon principios que siguen vigentes: el mínimo privilegio (cada componente debe operar con los permisos mínimos necesarios), la mediación

completa (toda operación debe verificarse contra la política de acceso), los valores por defecto seguros y la economía de mecanismo. La defensa en profundidad añade que ninguna capa debe ser la única barrera. La guía NIST SP 800-190 traslada estos principios a contenedores: imágenes mínimas, permisos estrictos sobre archivos sensibles, retiro de capacidades, límites de recursos y aislamiento de red (Souppaya et al., 2017).

2.7 Metodologías iterativas e incrementales

Frente a los modelos secuenciales, los enfoques iterativos e incrementales construyen el sistema en ciclos cortos, cada uno con un entregable verificable, lo que permite descubrir problemas temprano y ajustar el diseño con evidencia (Sommerville, 2011; Pressman y Maxim, 2015). El concepto de producto mínimo viable (MVP) propone entregar primero el núcleo funcional imprescindible y ampliarlo con base en lo aprendido (Ries, 2011). Estos ciclos cortos con un entregable verificable comparten, además, el espíritu del Manifiesto Ágil de priorizar el software funcionando por sobre la documentación exhaustiva (Beck et al., 2001), aun cuando el proyecto no adoptó un marco ágil formal. Este proyecto de ciberseguridad e infraestructura de contenedores combina el desarrollo iterativo con el enfoque MVP, que permite validar rigurosamente el núcleo de aislamiento, contención por kernel y orquestación básica antes de escalar hacia el catálogo completo de retos y la interfaz de administración.

Capítulo 3. Análisis de Vulnerabilidad

El análisis de vulnerabilidad se realizó en dos planos. El primero corresponde a las vulnerabilidades pedagógicas: las fallas deliberadas que cada reto expone para que el estudiante las explote. El segundo corresponde a las vulnerabilidades de la plataforma: los vectores por los que un estudiante malintencionado podría atacar al servidor, a otros estudiantes o a la integridad de los retos.

3.1 Modelo de amenazas de la plataforma

El modelado de amenazas identifica de forma sistemática los activos, los actores y los vectores de ataque antes de diseñar las contramedidas (Shostack, 2014). Siguiendo ese enfoque, los activos a proteger en esta plataforma son el servidor anfitrión (memoria, CPU, disco, socket de Docker), el entorno de cada estudiante, la base de datos de la plataforma y la integridad de las *flags*. El actor principal es un estudiante autenticado con acceso a una consola dentro de un contenedor; se asume que puede ejecutar cualquier comando disponible en la imagen y que puede actuar de forma hostil. No se considera un atacante externo sin cuenta, porque toda la API exige autenticación de Django. La Tabla 1 resume los vectores identificados y la contramedida prevista para cada uno.

Tabla 1*Vectores de ataque contra la plataforma identificados en el análisis*

Categoría	Vector	Impacto potencial	Contramedida diseñada
Recursos	Reserva de memoria sin freno	Agotar la RAM del host	Memory 256 MB (cgroup)
Recursos	Bucles de CPU concurrentes	Degradar a otros usuarios	NanoCpus 0,5 núcleos
Recursos	<i>Fork bomb</i>	Agotar la tabla de procesos	PidsLimit 64 + Init: true
Recursos	Apertura masiva de archivos	Agotar descriptores del host	Ulimits nofile 1024/2048
Recursos	Llenar el disco	Inutilizar el host	/tmp en tmpfs de 64 MB; /app de solo lectura por permisos (root:root, 0755)
Recursos	Inundar el log del contenedor	Llenar el disco del host	LogConfig 10 MB × 3
Red	Alcanzar el contenedor de otro usuario	Robo de <i>flags</i> , sabotaje	Red Internal por usuario
Red	Salir a Internet	Exfiltración, descarga de herramientas	Sin gateway (Internal: true)
Red	Alcanzar servicios del host	Atacar la propia plataforma	Sin ruta por defecto; sin docker.sock montado
Escalada	Obtener root vía capabilities	Escape del contenedor	CapDrop: ALL + CapAdd mínimo
Escalada	Injectar módulos vía PYTHONPATH en sudo	Ejecutar código como root	env_reset de sudo
Escalada	Crear <i>user namespace</i>	Ampliar superficie del kernel	Perfil <i>seccomp</i> por defecto de Docker
Integridad	Leer la <i>flag</i> directamente	Anular el reto	Archivo root:600 + sudo restringido
Ciclo de vida	Mantener la instancia viva con salida periódica	Ocupar recursos indefinidamente	Vida máxima absoluta
Ciclo de vida	Dejar procesos huérfanos al cerrar la consola	Agotar PidsLimit propio	Terminación por sesión al desconectar

3.2 Vulnerabilidades pedagógicas: los diez retos OWASP

Cada reto es una pequeña aplicación de consola escrita en Python que contiene una falla real de la categoría OWASP correspondiente. La Tabla 2 resume la vulnerabilidad, su

debilidad CWE asociada y la técnica de explotación esperada.

Tabla 2

Catálogo de retos, categoría OWASP, debilidad CWE y técnica de explotación

Reto (slug)	OWASP 2021	Dificultad	Vulnerabilidad y explotación
injection-sqli	A03 Inyección	Básico	Consulta SQL por concatenación (CWE-89). <i>Bypass</i> del login con <code>admin' --</code> .
idor	A01 Control de acceso	Básico	CLI de pedidos sin verificar propietario (CWE-639). Solicitar el pedido 1337 ajeno.
crypto-debil	A02 Fallas criptográficas	Básico	Contraseñas con MD5 sin sal (CWE-328). Ataque de diccionario con la <i>wordlist</i> provista.
misconfig	A05 Configuración incorrecta	Básico	Respaldo <code>config.yaml.bak</code> legible (CWE-538). Leer credenciales expuestas.
diseno-inseguro	A04 Diseño inseguro	Intermedio	Cupón sin límite de usos por compra (CWE-840). Repetir el cupón en una sola invocación.
componente-vulnerable	A06 Componentes vulnerables	Intermedio	<code>libcal 1.4.2</code> usa <code>eval()</code> sin sandbox (CWE-95). Inyectar una expresión Python.
auth-fallas	A07 Autenticación	Intermedio	PIN de 4 dígitos sin límite de intentos (CWE-307). Fuerza bruta de 10 000 combinaciones.
logging-fallas	A09 Registro y monitoreo	Intermedio	Token de sesión en texto plano en el log (CWE-532). Reutilizar el token.
integridad-datos	A08 Integridad de datos	Difícil	<code>pickle.load()</code> sobre archivo editable (CWE-502). Deserialización con carga maliciosa.
ssrf	A10 SSRF	Difícil	Lista negra sensible a mayúsculas (CWE-918). Evadir el filtro variando el caso del host.

Capítulo 4. Diseño de la Solución

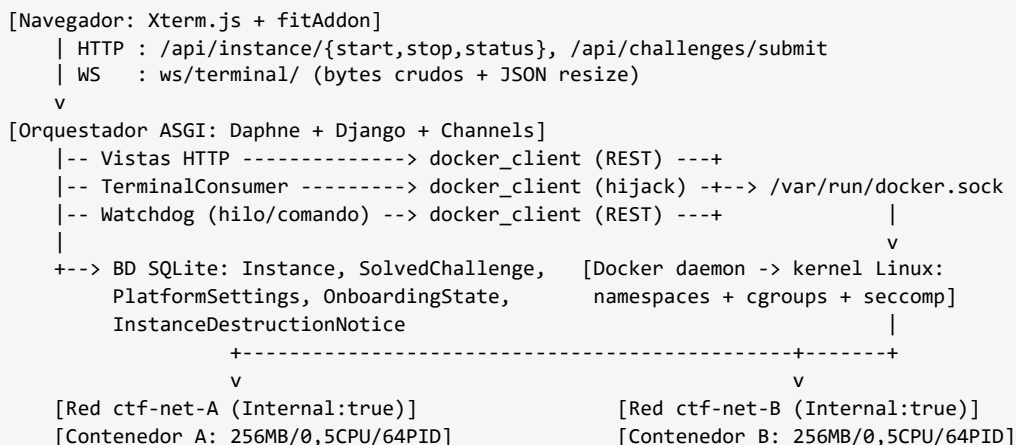
4.1 Arquitectura general

La solución se organiza en seis capas (Figura 1). La capa de presentación corre en el navegador y contiene Xterm.js. El orquestador ASGI, construido con Django, Channels y Daphne, atiende tanto las peticiones HTTP de control como las conexiones WebSocket de datos. El *driver* de Docker traduce esas operaciones a peticiones sobre el socket Unix. El daemon de Docker delega en el kernel la creación de namespaces y cgroups. El espacio de aislamiento contiene una red y un contenedor por usuario. Finalmente, el *watchdog* recorre la base de datos y destruye lo que venció.

Se separan de forma deliberada dos planos. El plano de control viaja por HTTP con JSON y protección CSRF: iniciar, detener y consultar el estado de la instancia, listar retos y enviar *flags*. El plano de datos viaja por un WebSocket binario: las pulsaciones de teclas van como bytes crudos hacia el proceso del contenedor, y los eventos de control, como el redimensionado, van como tramas JSON de texto.

Figura 1

Arquitectura general del sistema



4.2 Componentes

4.2.1 Driver de Docker (*docker_client.py*)

Es la única pieza que conoce la API de Docker. Para las operaciones de control usa requests-unixsocket contra los *endpoints* /networks/create, /containers/create, /containers/{id}/start, /containers/{id}/stop, /containers/{id}/exec y /exec/{id}/resize.

4.2.2 Vistas HTTP (*views.py*)

Exponen la API JSON bajo /api/: challenges/ devuelve el catálogo con el estado de resolución del usuario; instance/start/ crea red y contenedor; instance/stop/ los destruye; instance/status/ informa el reto activo, el tiempo restante y los umbrales; challenges/submit/ valida la *flag*.

Un segundo grupo bajo platform-settings/ da soporte al panel oculto de administración: configuración en vivo, instancias en vivo, destrucción manual y gestión de usuarios.

4.2.3 Consumidor WebSocket (*consumers.py*)

TerminalConsumer hereda de AsyncWebsocketConsumer. Al conectar, localiza la Instance del usuario, crea un exec con el *shell* adecuado y lanza una tarea lectora. Cada mensaje del navegador se reenvía al socket y actualiza last_activity. Al desconectar, cancela la tarea, cierra el socket y termina por sesión todos los procesos del exec.

4.2.4 Catálogo de retos (*challenges.py*)

Es un diccionario en memoria con los diez retos: nombre, categoría OWASP, dificultad, *flag* dinámica, imagen Docker, objetivo, primer paso y resultado esperado. El cliente solo envía el *slug*; la imagen real nunca viaja en la petición, lo que impide que un usuario solicite arrancar una imagen arbitraria del host.

4.2.5 Watchdog (*watchdog.py*, *apps.py*)

La función sweep_stale_instances() vive en un único módulo y la usan dos entradas: un hilo integrado que arranca junto al servidor y el comando manage.py watchdog. Recorre todas las instancias, calcula para cada una el tiempo inactivo y el tiempo de vida, y

destruye las que superan cualquiera de los dos límites. Si Docker no responde, conserva la fila para reintentar en la siguiente pasada, evitando dejar contenedores huérfanos.

4.3 Flujos principales

Inicio de instancia. El navegador envía `POST /api/instance/start/` con el *slug* del reto. La vista verifica que el usuario no tenga otra instancia, crea la red `ctf-net-{usuario}` con `Internal: true`, crea el contenedor con la imagen del catálogo y el `HostConfig` de la sección 4.4, lo arranca y guarda la fila `Instance`.

Sesión de terminal. Al abrir `ws/terminal/`, el consumidor crea el `exec`, realiza el *hijack* y mantiene dos flujos: del socket al WebSocket (salida del proceso) y del WebSocket al socket (teclas). Los mensajes JSON de tipo `resize` invocan `/exec/{id}/resize`.

Envío de flag. `POST /api/challenges/submit/` compara la cadena enviada con la bandera dinámica generada para ese usuario y ese reto. Si coincide y el usuario no había resuelto ese reto, crea `SolvedChallenge`. Cada intento fallido incrementa `failed_flag_attempts`; al quinto fallo la instancia se destruye.

Destrucción. Ocurre por tres vías: el botón Destruir (`POST /api/instance/stop/`), el *watchdog* (inactividad o vida máxima) y el panel de administración. En todos los casos se detiene y elimina el contenedor, se elimina la red y se borra la fila.

4.4 Mecanismos de aislamiento y endurecimiento

La configuración `HostConfig` común a los diez retos concentra las contramedidas del capítulo 3, tal como se muestra en la Figura 2.

Figura 2

Configuración de creación del contenedor

```
"HostConfig": {
  "NetworkMode": network_name,          # red Internal por usuario
  "Memory": 256 * 1024 * 1024,          # 256 MB
  "NanoCpus": 500_000_000,              # 0,5 núcleos
  "PidsLimit": 64,
  "CapDrop": ["ALL"],
  "CapAdd": ["SETUID", "SETGID"],        # solo lo que sudo necesita
  "Tmpfs": {"/tmp": "rw,noexec,nosuid,size=64m"},
  "Ulimits": [{"Name": "nofile", "Soft": 1024, "Hard": 2048}],
  "LogConfig": {"Type": "json-file", "Config": {"max-size": "10m", "max-file": "3"}},
  "Init": True,                          # tini como PID 1: cosecha zombies
}
```

No se aplica `ReadonlyRootfs` sobre el contenedor completo, porque el orquestador necesita inyectar la *flag* dinámica del estudiante dentro del contenedor antes de arrancarlo, mediante la Docker Engine API. La protección del sistema de archivos se sostiene entonces por permisos: `/app` y los binarios de cada reto pertenecen a `root:root` con modo `0755`, y el archivo con la *flag* tiene modo `600`, de manera que el usuario sin privilegios de la consola no puede escribir en `/app` ni leer la *flag* directamente.

A nivel de imagen, todos los retos heredan de `ctf-base` (Alpine con `bash`, `python3` y `sudo`). La consola corre como el usuario sin privilegios retador; la *flag* y los datos sensibles pertenecen a `root`; y un archivo en `/etc/sudoers.d/` autoriza sin contraseña exactamente el comando de la aplicación vulnerable. De este modo, la única vía hacia la *flag* es explotar la falla a través de ese comando.

4.5 Decisiones de diseño y alternativas descartadas

La Tabla 3 contrasta cada decisión arquitectónica con la alternativa que se descartó y con la razón que sostuvo esa elección.

Tabla 3*Decisiones arquitectónicas, alternativas descartadas y justificación*

Decisión	Alternativa descartada	Justificación
Socket Unix directo + <i>hijacking</i> manual	docker-py; subprocess con el binario docker	Cumple la restricción del proyecto sin crear un subproceso por usuario ni fugar descriptores.
ASGI (Daphne + Channels)	WSGI síncrono (Gunicorn)	Cada consola es una conexión de larga duración; WSGI bloquearía un <i>worker</i> por sesión.
Red Internal por usuario	Red compartida + reglas iptables	El kernel no crea <i>gateway</i> ; aislamiento sin tocar el firewall del host ni requerir root.
Límites por cgroups en HostConfig	Monitor en Python que mate procesos	El kernel actúa en tiempo real, durante la propia llamada al sistema.
<i>Watchdog</i> con estado en BD	threading.Timer en memoria	Sobrevive reinicios del servidor; no deja contenedores huérfanos.
loop.sock_recv() sin hilos	Hilo bloqueante por consola	Elimina el techo del pool de hilos y permite cancelación inmediata.
InMemoryChannelLayer y SQLite	Redis y PostgreSQL	Suficiente para un proceso y un despliegue de un solo host; el cambio está previsto como mejora futura.

Capítulo 5. Metodología de Desarrollo

5.1 Enfoque adoptado

El proyecto siguió una metodología iterativa e incremental orientada a un MVP, con iteraciones de un día y un entregable verificable al final de cada una (Sommerville, 2011). El punto de partida fue un Documento de Diseño de Software (SDD) que fijó objetivo, alcance, arquitectura, modelo de datos, flujos, decisiones de diseño y exclusiones explícitas. A partir de él se elaboró un plan de siete días para el MVP, como también para incorporar el catálogo OWASP, el panel de administración, la auditoría adversarial y, en una fase final, una suite de pruebas automatizadas.

5.2 Herramientas y entorno

El desarrollo se realizó sobre Ubuntu en WSL2 con Docker Desktop (Engine 28.5.1) y Python 3.11. Las dependencias se fijaron en `requirements.txt`: Django 5, Channels 4, Daphne 4, `requests-unixsocket` 0.3 y `urllib3`. El entorno virtual y la base SQLite se ubican en el sistema de archivos nativo de Linux.

Capítulo 6. Implementación y Pruebas

6.1 Stack tecnológico

La Tabla 4 detalla las tecnologías empleadas en cada capa del sistema y el papel que cumple cada una.

Tabla 4

Tecnologías empleadas por capa

Capa	Tecnología	Rol
Frontend	HTML, CSS, JavaScript, Xterm.js con fitAddon	Emulación de terminal, login, selección de retos, envío de <i>flags</i>
Servidor	Django 5, Django Channels 4, Daphne 4	Vistas HTTP, consumidor WebSocket, ASGI, ASGIStaticFilesHandler
Docker	requests-unixsocket, socket de la biblioteca estándar	REST de control y <i>hijacking</i> del exec
Persistencia	SQLite (migraciones de Django)	Instance, SolvedChallenge, PlatformSettings, OnboardingState, InstanceDestructionNotice
Retos	Docker, Alpine, Python 3, sudo	Diez imágenes derivadas de ctf-base
Sistema	Linux (WSL2), cgroups v2, namespaces, <i>seccomp</i>	Aislamiento y límites reales

6.2 Implementación de los componentes clave

6.2.1 Hijacking del exec

La implementación manual del protocolo de *hijack* fue un reto. El cliente envía sobre el socket Unix una petición como la de la Figura 3 y lee hasta el fin de las cabeceras; si el estado es 101, el resto del flujo son bytes.

Figura 3*Petición de hijack*

```
POST /exec/{exec_id}/start HTTP/1.1
Host: docker
Content-Type: application/json
Connection: Upgrade
Upgrade: tcp
Content-Length: ...

{"Detach": false, "Tty": true}
```

6.2.2 Doble criterio de destrucción

El *watchdog* evalúa dos condiciones independientes para cada instancia: inactividad y vida máxima. La vida máxima se introdujo tras notar que un bucle con salida periódica (`while true; do date; sleep 3; done &`) mantenía `last_activity` fresco indefinidamente.

6.2.3 Terminación por sesión

Al desconectar una consola, `terminate_console()` agrupa los procesos del contenedor por identificador de sesión y los termina, para no dejar procesos huérfanos consumiendo el cupo de `PidsLimit` del propio estudiante.

6.3 Estrategia y niveles de prueba

La verificación del sistema se organizó en cuatro niveles, de menor a mayor contacto con el entorno real, que se resumen en la Tabla 5. Los dos primeros responden si el sistema está bien construido; los dos últimos, si resiste el uso bajo condiciones adversas.

Tabla 5*Niveles de verificación aplicados al sistema*

Nivel	Qué verifica	Entorno
Unitario	Lógica del orquestador aislada: catálogo, banderas, modelos, API y <i>watchdog</i>	Docker sustituido por dobles de prueba
Humo	Ciclo de vida completo de una instancia contra el motor real	Docker Engine real
Estrés	Resistencia del aislamiento ante ataques ejecutados desde el contenedor	Docker real, dos configuraciones por prueba
Carga	Comportamiento del sistema con veinte estudiantes simultáneos	Servidor Daphne y veinte contenedores reales

En el nivel de estrés, cada ataque se ejecuta dos veces: primero sobre un contenedor sin los límites de la plataforma y después sobre uno con la configuración real descrita en la sección 4.4. De este modo la contención que ofrece el kernel queda medida por contraste y no solamente afirmada. La campaña completa se ejecutó sobre un equipo con 12 núcleos y 7,6 GB de RAM, bajo WSL2 con Docker Engine 28.5.1.

6.4 Pruebas unitarias automatizadas

Se construyó una suite de ochenta y cuatro casos en el paquete `ctf/tests/`, organizada en cinco módulos. Ninguno requiere Docker: las llamadas al motor de contenedores se sustituyen por dobles de prueba (*mocks*), de modo que la suite se ejecuta en cualquier equipo, sin depender de un daemon de Docker disponible, y puede incorporarse a un flujo de integración continua. La Tabla 6 indica el riesgo que cubre cada módulo.

Tabla 6*Módulos de la suite unitaria y riesgo que cubre cada uno*

Módulo	Riesgo que cubre
<code>test_challenges</code>	Que un estudiante acredite un reto sin explotar la vulnerabilidad, o que el catálogo público filtre una bandera
<code>test_views</code>	Que la API atienda a un usuario no autenticado, o que un fallo de Docker a mitad del despliegue deje recursos sin liberar
<code>test_seguridad</code>	Que una modificación posterior elimine un límite de recursos sin que nadie lo advierta
<code>test_models</code>	Que un estudiante acumule más de una instancia o duplique su progreso
<code>test_watchdog</code>	Que las instancias abandonadas no se liberen, o que un fallo de Docker deje contenedores huérfanos

El módulo `test_seguridad` cumple una función particular: intercepta la petición que el orquestador envía al socket de Docker y comprueba que contenga los límites de memoria, procesador, procesos, descriptores, capacidades y montajes descritos en la sección 4.4. Opera como salvaguarda ante regresiones, porque un cambio que suprima alguno de esos parámetros (por ejemplo, `PidsLimit` o la red `Internal`) hace fallar la suite antes de llegar a producción. El módulo `test_views` verifica además la regla de negocio más delicada de la API: si Docker falla a mitad de un despliegue, la vista debe eliminar cualquier red o contenedor que ya se hubiera creado, para no dejar recursos huérfanos sin una fila que los reclame; y el módulo `test_watchdog` verifica la contraparte de esa misma regla, que si el fallo ocurre durante la destrucción, la fila debe conservarse para reintentar en el siguiente barrido.

6.5 Pruebas de aislamiento

Antes de las diez pruebas de aislamiento, se verificó el ciclo de vida completo de una instancia contra el motor real de Docker, sin dobles de prueba: creación de la red interna, creación y arranque del contenedor, inyección de la bandera dinámica del estudiante, lectura de los límites desde la interfaz de `cgroups` del propio contenedor, apertura del `exec` de la consola, aparición del contenedor en el inventario que usa el panel de administración, y destrucción sin residuos en el host. Las diez comprobaciones resultaron correctas.

La Tabla 7 recoge las diez pruebas de aislamiento y de límites de recursos ejecutadas contra un contenedor de estudiante.

Tabla 7

Pruebas de aislamiento y de límites de recursos

N.º	Prueba	Resultado
1–2	Dos usuarios con instancias simultáneas, red y contenedor propios	Correcto; redes separadas
3	Alcanzar el contenedor del otro usuario (<i>ping</i> , <i>wget</i> , TCP directo, ruta manual)	Bloqueado: Network unreachable
4	Escanear vecinos de la propia red	Solo el propio contenedor
5	Salida a Internet (<i>wget</i> http://1.1.1.1)	Bloqueado: sin ruta por defecto
6	Alcanzar servicios del host y la plataforma	Sin servicios alcanzables; <code>docker.sock</code> no montado
7	Reservar memoria sin freno	Contenido: contenedor vivo
8	Cuatro bucles infinitos de CPU	Contenido: uso topado en 49–55 %
9	<i>Fork bomb</i> :(){ : :& };;:	Contenido: can't fork, host intacto
10	Estado del host y del testigo	Sin degradación: 8–19 ms en todo momento

6.6 Pruebas de estrés y de carga

Las pruebas de estrés repiten ocho de los ataques anteriores en dos configuraciones (sin los límites de la plataforma y con ellos) para medir la contención por contraste. Se ejecutaron sobre la imagen base del catálogo. La Tabla 8 presenta el comparativo.

Tabla 8

Comparativo de las pruebas de estrés, sin límites y con la configuración de la plataforma

Ataque	Sin los límites	Con la plataforma	Qué lo impide
Bomba de procesos (200 solicitados)	200 procesos creados	62 procesos; bloqueado por el kernel	cgroup pids (PidsLimit 64)
Agotar memoria (pedir 400 MB)	400 MB asignados	Detenido con código 137 (OOM)	cgroup memory + tope de swap
Saturar CPU (bucle infinito)	100,25 % de uso	50,02 % de uso	Planificador CFS (NanoCpus)
Salir a Internet	Conectado a 1.1.1.1	Network unreachable	Red Internal
Alcanzar a otro estudiante	Servicio ajeno visible	Network unreachable	Una red por usuario
Privilegios de root	14 capacidades activas; chown permitido	2 capacidades activas; chown denegado	CapDrop: ALL
Descriptores de archivo	1 048 576 disponibles	1 024 disponibles	Ulimits nofile
Perfil completo aplicado	—	6 de 6 límites verificados juntos	cgroups + namespaces

En los ocho casos, quien detiene el ataque es el kernel de Linux, no código de la aplicación: el planificador y los *cgroups* deciden durante la propia llamada al sistema, antes de conceder el recurso, lo que confirma en la práctica el principio de mínimo privilegio descrito en la sección 2.6.

La prueba de carga somete al sistema a veinte estudiantes simultáneos que se autentican, despliegan su instancia, abren la consola, ejecutan una orden y liberan los recursos, contra un servidor Daphne real y veinte contenedores verdaderos. La Tabla 9 registra la medición antes y después de la corrección.

Tabla 9*Prueba de carga con veinte consolas simultáneas, antes y después de la corrección*

Medición	Antes	Después
Consolas que ejecutaron el comando	0 / 20	20 / 20
Hilos del servidor	16 (tope del <i>pool</i>)	14
Estado de los hilos	unix_stream_read_generic (bloqueados)	futex_wait_queue (ociosos)
Sockets a Docker sin consolas abiertas	25 (filtrados)	3
POST /api/instance/start/	Timeout a los 30 s	14 ms
Fugas de procesos en 25 reconexiones	1 por reconexión	0

La campaña de esta sesión repitió la prueba de carga en un equipo distinto, con veinte estudiantes simulados desplegando el reto *injection-sqli*. Las veinte instancias se crearon correctamente en 7,79 segundos (latencia media de 5,71 s por instancia) y las veinte consolas WebSocket ejecutaron la orden enviada, con una latencia media de 1,63 s. La medición de hilos del servidor, tomada tanto durante el arranque de las consolas como con las veinte ya abiertas y en reposo, no registró ningún hilo bloqueado en entrada o salida de socket: los diecisiete hilos presentes permanecieron en estado *futex_wait_queue*, es decir, ociosos. El dato resulta concluyente porque ese equipo dispone de doce núcleos, de modo que el *pool* de hilos de Python admite un máximo de dieciséis; si cada consola retuviera un hilo propio, veinte sesiones simultáneas serían aritméticamente imposibles. Al finalizar, las veinte instancias se liberaron en 12,40 segundos y no quedó ninguna fila huérfana en la base de datos.

Capítulo 7. Resultados

La Tabla 10 contrasta cada objetivo específico con la evidencia que respalda su cumplimiento.

Tabla 10

Cumplimiento de los objetivos específicos

Objetivo específico	Estado	Evidencia
Cliente de la Docker Engine API sin SDK, con <i>hijacking</i>	Cumplido	<code>docker_client.py</code> : REST sobre socket Unix y HijackedExecSocket sin hilos
Orquestador Django con estado persistente	Cumplido	Cinco modelos (Instance, SolvedChallenge, PlatformSettings, OnboardingState, InstanceDestructionNotice); nueve migraciones
Consola Xterm.js con WebSocket asíncrono	Cumplido	TerminalConsumer; 20/20 consolas simultáneas verificadas en dos campañas; <i>bracketed paste</i> ; reconexión sin fugas
Aislamiento de red y límites de recursos	Cumplido	Pruebas concluidas
<i>Watchdog</i> por inactividad y vida máxima	Cumplido	Destrucción verificada mediante el módulo <code>test_watchdog</code>
Diez retos OWASP con separación de privilegios y puntaje	Cumplido	10/10 verificados de punta a punta; validación de bandera dinámica en servidor
Verificación empírica de aislamiento, límites y robustez	Cumplido	Realizadas pruebas unitarias, de humo, de estrés y de carga

7.1 Resultados cuantitativos

- Creación de instancia: ~0,87 s por usuario (0,77–1,01 s) en la medición original; 5,71 s de media en la campaña de carga de veinte usuarios simultáneos de esta sesión.
- Consolas simultáneas: 20 de 20 funcionales en dos campañas independientes, sin techo por hilos; cero hilos bloqueados en entrada o salida de socket con las veinte consolas abiertas.
- Latencia del host bajo ataque simultáneo de tres usuarios: 8–19 ms, sin degradación.

- Límites verificados: 268 435 456 B de memoria, 49–55 % de CPU, 64 procesos, ~1024 descriptores, 64 MB en /tmp.
- Fugas de procesos en 25 reconexiones: 0. Huérfanos tras la corrida completa de retos: 0.
- Cobertura OWASP: 10 de 10 categorías, con dificultad y guía por reto.
- Suite automatizada: 84 casos unitarios ejecutados en 8,35 s, sin fallos ni dependencia de Docker.
- Verificación contra Docker real: 10 de 10 pruebas de humo correctas, sin residuos en el host tras la destrucción.
- Contención de estrés medida por contraste: 200 procesos sin límite frente a 62 con él; 100,25 % de CPU frente a 50,02 %; salida a Internet y acceso al entorno de otro estudiante bloqueados en ambos casos con la configuración de la plataforma.

7.2 Resultados cualitativos

El sistema cumple los tres retos planteados: *backend* sin SDK, *frontend* con terminal real y *modo difícil* con contención por kernel y destrucción automática. Más allá de lo exigido, incorporó un catálogo completo alineado a OWASP, configuración en vivo y un panel de instancias en vivo también. El proceso dejó como subproducto una documentación extensa.

7.3 Defectos detectados por la campaña de validación

La campaña de pruebas de esta sesión no se limitó a confirmar el comportamiento esperado: detectó tres defectos que la configuración declarada de la plataforma ocultaba. Ninguno de los tres compromete el aislamiento entre estudiantes verificado en la sección 6.5; los tres afectan la capacidad real del sistema o el aprovechamiento de sus recursos. La Tabla 11 detalla cada defecto, la evidencia medida y la corrección propuesta.

Tabla 11*Defectos detectados, evidencia medida y corrección propuesta*

Defecto	Evidencia medida	Corrección propuesta
El techo de instancias simultáneas lo impone la red de Docker, no la memoria del host	Al elevar la prueba de carga a treinta estudiantes, seis despliegues fallaron con error 500. Docker agota sus rangos de direcciones predefinidos alrededor de la red número veinticuatro: <code>all predefined address pools have been fully subnetted</code>	Declarar un rango de direcciones propio en <code>daemon.json</code> (por ejemplo, base <code>10.200.0.0/16</code> con máscara <code>/24</code> , que habilita 256 redes)
El límite de memoria efectivo por contenedor duplica al declarado	Con el límite de 256 MB vigente, un contenedor asignó 400 MB sin ser detenido; al declarar también el tope de <code>swap</code> , el kernel detuvo la asignación con código 137	Añadir <code>"MemorySwap"</code> : <code>mem_limit_bytes</code> a la configuración de creación del contenedor
La destrucción de cada instancia desperdicia cinco segundos	El proceso principal del contenedor no atiende <code>SIGTERM</code> , de modo que Docker agota el plazo de gracia completo antes de forzar el cierre: 5,30 s frente a 0,37 s sin esa espera	Reducir el plazo de gracia en <code>stop_container()</code>

Capítulo 8. Conclusiones y Mejoras Futuras

8.1 Conclusiones

Primero, es posible construir un orquestador de contenedores funcional y seguro hablando directamente con la Docker Engine API. La restricción de no usar SDK, lejos de ser un obstáculo artificial, obligó a comprender el protocolo de *hijacking* y condujo a una implementación más ligera que la alternativa por subprocesos.

Segundo, el aislamiento efectivo en un entorno multiusuario no proviene de una sola medida sino de la superposición de varias: red interna sin *gateway*, *cgroups* para memoria, CPU y procesos, retiro de capacidades, permisos estrictos sobre los datos sensibles, límites de descriptors y un `init` real. La campaña de pruebas de estrés mostró que cada capa cubre un vector que las demás no cubren, en línea con el principio de defensa en profundidad (Saltzer y Schroeder, 1975; Souppaya et al., 2017).

Tercero, la verificación empírica descubrió defectos que la configuración declarada ocultaba: el techo del *pool* de hilos, la instancia inmortal por salida periódica, el *bypass* de los retos con `cat`, los zombis de la *fork bomb*, los huérfanos al cerrar la consola y, en la campaña más reciente, el límite real de memoria duplicado por el *swap* y el techo de concurrencia impuesto por las redes de Docker. Ninguno de ellos habría aparecido sin atacar el sistema de forma deliberada. Esta conclusión respalda la práctica de cerrar cada iteración con pruebas que intenten romper lo construido, y de automatizar esas pruebas para que se repitan sin esfuerzo adicional.

Cuarto, la integridad pedagógica de un CTF depende de la separación de privilegios dentro del contenedor. El patrón adoptado (usuario sin privilegios, dato sensible de `root` con modo 600 y `sudo` restringido al comando exacto) es reutilizable para cualquier reto futuro, y quedó verificado tanto manualmente como por la suite automatizada del módulo `test_challenges`.

Quinto, la metodología iterativa con reportes diarios produjo una trazabilidad completa entre requisitos, decisiones, pruebas y correcciones, lo que facilitó tanto el desarrollo como la elaboración de este documento.

8.2 Limitaciones

El sistema se diseñó y se validó como un despliegue de un solo anfitrión, con un único proceso de servidor, un único daemon de Docker y un archivo SQLite. De ese supuesto se derivan las limitaciones siguientes.

Primero, la concurrencia real está acotada por los rangos de direcciones que Docker asigna a sus redes antes que por la memoria del equipo.

Segundo, no existe un límite global de instancias a nivel de plataforma, sino únicamente la restricción de una instancia activa por usuario. El total de contenedores desplegados a la vez es, en la práctica, lo que el anfitrión resista.

Tercero, `InMemoryChannelLayer` y el hilo integrado del *watchdog* impiden ejecutar más de un proceso de servidor sin trabajo adicional de configuración, lo que limita el escalado horizontal de la capa web.

8.3 Mejoras futuras

1. **Ampliar el espacio de direcciones de Docker y fijar un límite global de instancias.** La configuración predeterminada del daemon agota sus rangos de red alrededor de la instancia número veinticuatro (sección 7.3), lo que acota la concurrencia por debajo de lo que permitiría la memoria disponible. Declarar un rango de direcciones propio en `daemon.json` eleva ese techo a 256 redes; hecho eso, conviene fijar además un límite global de instancias simultáneas, del que hoy se carece.
2. **PostgreSQL.** SQLite serializa cada escritura y cada tecla actualiza `last_activity`; `InMemoryChannelLayer` no escala a varios procesos. Ambos cambios están anticipados en el SDD.
3. **Perfil *seccomp* personalizado.** Hoy se usa el perfil por defecto de Docker. Un perfil más estricto reduciría aún más la superficie del kernel expuesta.
4. **Manejo de errores cuando Docker no responde.** El error 500 actual es crudo; conviene detectar el estado del daemon y comunicarlo con claridad al usuario.
5. **Ranking y métricas de aprendizaje.** El modelo `SolvedChallenge` permite construir tablas de posiciones, tiempos de resolución y estadísticas por categoría

OWASP para el docente.

6. **Instancias múltiples por usuario.** Sustituir la relación uno a uno por una relación uno a muchos con un tope configurable.

Referencias

- Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Kern, J., Marick, B., Martin, R. C., Mellor, S., Schwaber, K., Sutherland, J., & Thomas, D. (2001). *Manifesto for Agile Software Development*. <https://agilemanifesto.org/>
- Castellanos, D., Pérez, S., Guillén, A., Agostini, P. y Peña, J. (2026). *Orquestador-daemon: Plataforma CTF con contenedores dinámicos* [Código fuente y documentación]. GitHub. <https://github.com/Daniel132005/Orquestador-daemon>
- Chapman, P., Burket, J., & Brumley, D. (2014). PicoCTF: A game-based computer security competition for high school students. En *Proceedings of the 2014 USENIX Summit on Gaming, Games, and Gamification in Security Education (3GSE 14)*. USENIX Association. <https://www.usenix.org/conference/3gse14/summit-program/presentation/chapman>
- Combe, T., Martin, A., & Di Pietro, R. (2016). To Docker or not to Docker: A security perspective. *IEEE Cloud Computing*, 3(5), 54–62. <https://doi.org/10.1109/MCC.2016.100>
- Django Software Foundation. (2024a). *Django Channels documentation* (Versión 4). <https://channels.readthedocs.io/>
- Django Software Foundation. (2024b). *ASGI (Asynchronous Server Gateway Interface) specification* (Versión 3.0). <https://asgi.readthedocs.io/>
- Django Software Foundation. (2024c). *Django documentation* (Versión 5.0). <https://docs.djangoproject.com/en/5.0/>
- Docker Inc. (2024a). *Docker Engine API reference* (Versión 1.45). <https://docs.docker.com/engine/api/>
- Docker Inc. (2024b). *Docker Engine security*. <https://docs.docker.com/engine/security/>
- Fette, I., & Melnikov, A. (2011). *The WebSocket Protocol* (RFC 6455). Internet Engineering Task Force. <https://doi.org/10.17487/RFC6455>
- Kerrisk, M. (2024a). *namespaces(7) — Linux manual page*. <https://man7.org/linux/man-pages/man7/namespaces.7.html>

- Kerrisk, M. (2024b). *cgroups(7) — Linux manual page*. <https://man7.org/linux/man-pages/man7/cgroups.7.html>
- Kerrisk, M. (2024c). *capabilities(7) — Linux manual page*. <https://man7.org/linux/man-pages/man7/capabilities.7.html>
- Merkel, D. (2014). Docker: Lightweight Linux containers for consistent development and deployment. *Linux Journal*, 2014(239), Artículo 2.
- MITRE Corporation. (2023). *Common Weakness Enumeration (CWE)* (Versión 4.13). <https://cwe.mitre.org/>
- OWASP Foundation. (2021). *OWASP Top 10:2021*. <https://owasp.org/Top10/>
- Pressman, R. S., & Maxim, B. R. (2015). *Software engineering: A practitioner's approach* (8.^a ed.). McGraw-Hill Education.
- Ries, E. (2011). *The lean startup: How today's entrepreneurs use continuous innovation to create radically successful businesses*. Crown Business.
- Saltzer, J. H., & Schroeder, M. D. (1975). The protection of information in computer systems. *Proceedings of the IEEE*, 63(9), 1278–1308. <https://doi.org/10.1109/PROC.1975.9939>
- Shostack, A. (2014). *Threat modeling: Designing for security*. Wiley.
- Sommerville, I. (2011). *Ingeniería de software* (9.^a ed.). Pearson Educación.
- Souppaya, M., Morello, J., & Scarfone, K. (2017). *Application container security guide* (NIST Special Publication 800-190). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.SP.800-190>
- Sultan, S., Ahmad, I., & Dimitriou, T. (2019). Container security: Issues, challenges, and the road ahead. *IEEE Access*, 7, 52976–52996. <https://doi.org/10.1109/ACCESS.2019.2911732>
- Vigna, G. (2003). Teaching hands-on network security: Testbeds and live exercises. *Journal of Information Warfare*, 2(3), 8–24.
- Xterm.js Contributors. (2024). *xterm.js: A terminal for the web*. <https://xtermjs.org/>

Anexos

Anexo A

Pruebas unitarias

● ● ● ejecución completa

pruebas/resultados/01-unitarias.txt

```
$ python manage.py test ctf.tests -v 2
```

```
Creating test database for alias 'default'...
```

```
Found 84 test(s).
```

```
[... ejecución de los cinco módulos ...]
```

```
-----  
Ran 84 tests in 8.351s
```

```
OK
```

```
Destroying test database for alias 'default'...
```

```
System check identified no issues (0 silenced).
```

Anexo B

Catálogo de retos y banderas dinámicas

```
● ● ● test_challenges · 20 casos

$ python manage.py test ctf.tests.test_challenges -v 2

CatalogoRetosTest - integridad del catálogo
  test_el_catalogo_tiene_diez_retos ... ok
  test_cada_reto_cubre_una_categoria_owasp_distinta ... ok
  test_todos_los_retos_declaran_los_campos_obligatorios ... ok
  test_todas_las_dificultades_son_validas ... ok
  test_el_reto_por_defecto_existe_en_el_catalogo ... ok
  test_get_challenge_devuelve_none_si_no_existe ... ok

BanderaDinamicaTest - generación de la bandera
  test_la_bandera_es_determinista_para_el_mismo_usuario ... ok
  test_la_bandera_cambia_entre_usuarios ... ok
  test_la_bandera_cambia_entre_retos_del_mismo_usuario ... ok
  test_la_bandera_respetar_el_formato_publicado ... ok
  test_todos_los_retos_generan_banderas_distintas_entre_si ... ok

ValidacionBanderaTest - quién puede canjearla
  test_acepta_la_bandera_propia ... ok
  test_rechaza_la_bandera_de_otro_estudiante ... ok
  test_rechaza_la_bandera_correcta_de_otro_reto ... ok
  test_rechaza_si_no_se_identifica_al_usuario ... ok
  test_rechaza_bandera_vacia ... ok
  test_rechaza_reto_inexistente ... ok
  test_ignora_espacios_alrededor_de_la_bandera ... ok

ExposicionDeDatosTest - qué se publica al frontend
  test_list_challenges_no_expone_ninguna_bandera ... ok
  test_list_challenges_devuelve_todos_los_retos_con_xp ... ok

Ran 20 tests - OK
```

Anexo C

Invariantes de la base de datos

● ● ● test_models · 13 casos

```
$ python manage.py test ctf.tests.test_models -v 2
```

InstanceTest – una instancia activa por estudiante

```
test_un_usuario_no_puede_tener_dos_instancias ... ok
test_dos_usuarios_distintos_pueden_tener_instancia_a_la_vez ... ok
test_el_contador_de_intentos_arranca_en_cero ... ok
test_borrar_el_usuario BORRA su instancia ... ok
```

PlatformSettingsTest – configuración en caliente

```
test_actual_crea_la_fila_si_no_existe ... ok
test_actual_devuelve_siempre_la_misma_fila ... ok
test_limite_de_tiempo_por_dificultad ... ok
test_dificultad_desconocida_usa_el_limite_basico ... ok
test_el_cambio_se_persiste_sin_reiniciar ... ok
```

SolvedChallengeTest – progreso y XP

```
test_no_se_puede_registrar_dos_veces_el_mismo_reto ... ok
test_dos_usuarios_pueden_resolver_el_mismo_reto ... ok
```

InstanceDestructionNoticeTest – aviso del motivo

```
test_los_tres_motivos_estan_disponibles ... ok
test_se_puede_registrar_un_aviso ... ok
```

Ran 13 tests – OK

Anexo D

API HTTP y ciclo de vida de la instancia

```
● ● ● test_views · 23 casos

$ python manage.py test ctf.tests.test_views -v 2

AutenticacionRequeridaTest - nada es público
  test_get_sin_sesion_no_devuelve_datos ... ok
  test_post_sin_sesion_no_ejecuta_nada ... ok
  test_la_terminal_no_es_publica ... ok

CatalogoAPITest - qué devuelve /api/challenges/
  test_devuelve_los_diez_retos ... ok
  test_no_filtra_ninguna_bandera_al_frontend ... ok

DespliegueInstanciaTest - POST /api/instance/start/
  test_despliega_y_registra_la_instancia ... ok
  test_inyecta_la_bandera_del_estudiante_en_el_contenedor ... ok
  test_la_red_es_exclusiva_del_usuario ... ok
  test_rechaza_una_segunda_instancia_del_mismo_usuario ... ok
  test_rechaza_un_reto_inexistente ... ok
  test_si_docker_falla_no_deja_recursos_colgados ... ok

DetenerInstanciaTest - POST /api/instance/stop/
  test_destruye_contenedor_red_y_fila ... ok
  test_sin_instancia_activa_responde_404 ... ok
  test_si_docker_falla_conserva_la_fila ... ok

EnvioBanderaTest - POST /api/challenges/submit/
  test_bandera_correcta_registra_el_reto_resuelto ... ok
  test_bandera_de_otro_estudiante_no_sirve ... ok
  test_bandera_vacia_se_rechaza_sin_gastar_intento ... ok
  test_resolver_dos_veces_no_duplica_el_progreso ... ok
  test_cada_fallo_descuenta_un_intento ... ok
  test_al_quinto_fallo_destruye_la_instancia ... ok
  test_redesplegar_reinicia_el_contador_de_intentos ... ok
  test_reto_inexistente_responde_404 ... ok
  test_json_invalidado_se_rechaza ... ok

Ran 23 tests - OK
```

Anexo E

Límites enviados al kernel y panel de administración

```
● ● ● test_seguridad · 16 casos

$ python manage.py test ctf.tests.test_seguridad -v 2

LimitesDeRecursosTest - lo que el contenedor podrá consumir
test_declara_limite_de_memoria ... ok
test_declara_limite_de_cpu ... ok
test_declara_limite_de_procesos ... ok
test_declara_limite_de_descriptores_de_archivo ... ok
test_el_contenedor_usa_la_red_del_usuario ... ok

EndurecimientoContenedorTest - lo que NO podrá hacer
test_descarta_todas_las_capacidades_del_kernel ... ok
test_solo_devuelve_las_capacidades_minimas_para_sudo ... ok
test_tmp_esta_en_memoria_sin_ejecucion_ni_setuid ... ok
test_usa_un_init_que_cosecha_procesos_zombis ... ok
test_acota_el_tamano_de_los_logs_en_disco ... ok

AislamientoDeRedTest - la red de cada estudiante
test_la_red_es_interna_sin_salida_a_internet ... ok
test_la_red_es_de_tipo_bridge ... ok

PanelAdministracionTest - separación de roles
test_un_estudiante_no_accede_a_la_configuracion ... ok
test_un_estudiante_no_puede_destruir_instancias_ajenas ... ok
test_un_anonimo_tampoco_accede ... ok
test_el_profesor_si_accede_a_la_configuracion ... ok

Ran 16 tests - OK
```

Anexo F

Destrucción automática de instancias

```
● ● ● test_watchdog · 12 casos

$ python manage.py test ctf.tests.test_watchdog -v 2

Instancias que deben sobrevivir
  test_no_destruye_una_instancia_activa_y_reciente ... ok
  test_no_destruye_justo_antes_del_umbral ... ok

Expiración por abandono
  test_destruye_por_inactividad ... ok
  test_el_aviso_por_inactividad_lleva_ese_motivo ... ok

Expiración por vida máxima
  test_destruye_por_vida_maxima_aunque_este_activa ... ok
  test_la_vida_maxima_depende_de_la_dificultad ... ok
  test_reto_fuera_del_catalogo_usa_el_limite_global ... ok
  test_deja_aviso_con_el_motivo_correcto ... ok

Resistencia a fallos de Docker
  test_si_docker_falla_no BORRA la fila ... ok
  test_un_fallo_no impide limpiar las demas ... ok

Barrido masivo
  test_barre_varias_instancias_en_una_pasada ... ok
  test_barrido_sin_instancias_no falla ... ok

Ran 12 tests - OK
```

Anexo G

Pruebas de humo

```
pruebas de humo pruebas/resultados/02-humo.txt

$ python pruebas/humo.py

Docker Engine: real | Dobles de prueba: ninguno

[H-01] El cliente Docker propio crea una red aislada
$ docker_client.create_network('ctf-net-humo-1789949475')
Id de red: acb1b1e9250c
Tiempo: 0.036 s
Inspeccion: Internal=true Driver=bridge
>> RESULTADO: OK - red sin salida a internet

[H-02] Crea el contenedor del reto con limites de recursos
$ docker_client.create_container(image=ctf-reto-idor:latest, mem=256MB, cpus=0.5, pids=64)
Id de contenedor: 1166d8848825
Tiempo: 0.071 s
>> RESULTADO: OK

[H-03] Inyecta la bandera personalizada del estudiante
$ docker_client.inject_challenge_flag(cont, 'idor', <flag del alumno 1>)
Bandera generada: FLAG{idor_12f669e21f3b8de1}
>> RESULTADO: OK

[H-04] Arranca el contenedor
$ docker_client.start_container(cont)
Tiempo: 0.282 s
Estado del contenedor: running
>> RESULTADO: OK

[H-05] Los limites llegaron al kernel (vistos desde dentro)
$ docker exec <cont> cat /sys/fs/cgroup/memory.max /sys/fs/cgroup/pids.max
memory.max = 268435456 (esperado 268435456)
pids.max    = 64 (esperado 64)
cpu.max     = 50000 100000 (esperado = 0.5 CPU)
>> RESULTADO: OK - cgroups aplicados por el kernel

[H-06] La bandera NO es legible por el usuario sin privilegios
$ docker exec -u retador <cont> cat /app/pedidos.json
cat: can't open '/app/pedidos.json': Permission denied
Codigo de salida: 1
>> RESULTADO: OK - hay que explotar la falla para obtenerla

[H-07] La bandera SI es accesible explotando el reto (via sudo)
$ docker exec -u retador <cont> sudo /usr/bin/python3 /app/pedidos.py 1337
{
  "cliente": "admin",
  "producto": "Acceso VIP",
  "flag": "FLAG{idor_12f669e21f3b8de1}"
}
>> RESULTADO: OK - el reto es resoluble
```

```
[H-08] La consola interactiva puede abrirse (exec del TTY)
$ docker_client.create_exec(cont, ['/bin/sh'])
Id del exec: 8615f970ba47
>> RESULTADO: OK

[H-09] El contenedor aparece en el inventario de la plataforma
$ docker_client.list_ctf_containers()
Contenedores CTF vivos: 1 (Ids: 1166d8848825)
Imagen: ctf-reto-idor:latest | Estado: running | Red: ctf-net-humo-1789949475
>> RESULTADO: OK - el panel puede detectar huerfanos

[H-10] Destruccion completa: contenedor y red
$ docker_client.destroy_instance(cont, red)
Tiempo: 5.534 s
Contenedor restante: ninguno
Red restante: ninguna
>> RESULTADO: OK - sin residuos en el host

=====
Total: 10 | Exitosas: 10 | Fallidas: 0
```

Anexo H

Pruebas de estrés: bomba de procesos

● ● ● E-01 · bomba de procesos

--- ANTES · contenedor sin PidsLimit ---

```
$ docker run --rm ctf-base:latest python3 -c '<fork x200>'
```

procesos creados: 200

--- DESPUES · contenedor con --pids-limit=64 ---

```
$ docker run --rm --pids-limit=64 ctf-base:latest python3 -c '<fork x200>'
```

bloqueado por el kernel: [Errno 11] Resource temporarily unavailable

procesos creados: 62

COMPARATIVO | sin limite: 200 procesos -> con limite: 62 procesos

Anexo I

Un estudiante intenta alcanzar al otro

● ● ● E-05 · aislamiento entre estudiantes

Alumno B expone un servicio en 172.22.0.2:8000

--- ANTES · ambos alumnos en la MISMA red ---

\$ docker run --rm --network ctf-estres-abierta ... <conectar a IP del alumno B>

ALCANZADO: el alumno B es visible desde aqui

--- DESPUES · cada alumno en su PROPIA red interna ---

\$ docker run --rm --network ctf-estres-red-b ... <conectar a IP del alumno B>

INALCANZABLE: OSError [Errno 101] Network unreachable

Anexo J

Perfil completo de la plataforma

● ● ● E-08 · perfil aplicado

```
$ docker run --rm --memory=256m --cpus=0.5 --pids-limit=64 \
  --cap-drop=ALL --cap-add=SETUID --cap-add=SETGID \
  --tmpfs /tmp:rw,noexec,nosuid,size=64m \
  --ulimit nofile=1024:2048 --init ctf-base:latest sh -c '...'

memoria max : 268435456
cpu max      : 50000 100000
pids max     : 64
descriptores: 1024
capacidades : 000000000000000c0
montaje tmp  : tmpfs /tmp tmpfs rw,nosuid,nodev,noexec,relatime,size=65536k
```

Anexo K

Prueba de carga con veinte estudiantes

```

● ● ● prueba de carga pruebas/resultados/04-carga.txt

$ python pruebas/carga.py 20

[C-00] PREPARACION: CUENTAS Y ESTADO INICIAL
  Cuentas listas: 20 (nuevas: 0)
  Hilos del servidor en reposo: 1
  Sockets abiertos en reposo: 3
  Contenedores CTF vivos: 0

[C-01] AUTENTICACION CONCURRENTES DE 20 ESTUDIANTES
  Sesiones iniciadas: 20 / 20
  Tiempo total: 1.21 s

[C-02] DESPLIEGUE CONCURRENTES DE 20 INSTANCIAS
  Instancias creadas: 20 / 20
  Tiempo total de la oleada: 7.79 s
  Latencia por instancia -> min 1.71 s | media 5.71 s | max 7.89 s
  Filas en base de datos: 20  Contenedores reales en Docker: 20

[C-03] CONSOLAS WEBSOCKET SIMULTANEAS
  Consolas que ejecutaron el comando: 20 / 20
  Respuesta por consola -> min 1.31 s | media 1.63 s | max 1.88 s

[C-04] CONSUMO DE HILOS DEL SERVIDOR (metrica critica)
  $ for t in /proc/7936/task/*; do cat $t/wchan; done | sort | uniq -c

  T1 - durante el arranque de las consolas:
    17 futex_wait_queue      (ociosos)
    1  do_epoll_wait
    bloqueados en E/S de socket: 0

  T2 - con las 20 consolas ya abiertas y en reposo:
    17 futex_wait_queue      (ociosos)
    1  do_epoll_wait
    bloqueados en E/S de socket: 0

  Hilos totales: reposo 1 -> arranque 18 -> consolas abiertas 18

[C-05] CONSUMO REAL DE RECURSOS DEL HOST BAJO CARGA
  $ docker stats --no-stream
  great_solomon      0.00%   664KiB / 256MiB
  sharp_nightingale   0.00%   672KiB / 256MiB
  ... (20 contenedores activos)
  Consumo medio por contenedor: ~665 KiB

[C-06] LIBERACION DE RECURSOS
  Instancias detenidas: 20 / 20
  Tiempo total: 12.40 s
  Filas en base de datos tras la limpieza: 0
  Contenedores restantes: 0

```